

GPRS Network Prototype based on SDR and OpenBTS, as an IoT-lab Testbed.

Oscar Enrique Llerena Castro

Instituto Nacional de Investigación y Capacitación de Telecomunicaciones - INICTEL-UNI
Universidad Nacional de Ingeniería - Peru
 Email: ollerena@inictel-uni.edu.pe

Abstract—INICTEL-UNI research institute participates in various projects related to the design and implementation of Internet-of-Things applications in Peru. Regarding the different radio-access technologies that exist so the end-devices can reach the IoT cloud, the conclusion we share is that, even though several radio-access options are being rolled-out worldwide, GPRS continues to be the most available radio-access at remote zones, if not the only option in some cases. GPRS has a massive deployment due to 2G networks, and its data rate and latency meet the requirements for IoT applications like in smart agriculture or water pollution monitoring, for instance.

Given the necessity for counting with a GPRS network testbed to test our GPRS-based sensor node prototypes, at the lab, we managed to build a small-scale GPRS network based on the OpenBTS software platform and Software-Defined Radio hardware to employ it as a testbed for IoT application testing. In this paper, we briefly explain the concept of software-defined radio and provide a succinct description of the OpenBTS functionalities. Then, we share details related to the prototype implementation, configuration, and start-up as well as an end-device setup for testing. Finally, we share conclusions obtained from two experiments conducted over the network prototype. The first is to verify the GPRS signaling generated in data exchange between an end-device and an IoT server, and the second is to experiment with the accessibility rate of the network prototype when various end-devices share the communication channel.

Index Terms—IoT, M2M, GPRS, SDR, OpenBTS, USRP, radio-access.

I. INTRODUCTION

The INICTEL-UNI research institute is involved in projects that require the design and implementation of Internet of Things (IoT) and Machine-to-Machine (M2M) applications, mainly deployed in Peruvian rural areas [3]. IoT concept addresses the idea of connecting any object equipped with digital functionalities, capable of collecting and sending data to a server in the cloud, for post-analysis and without human intervention [1]. A type of these "objects" certainly is the Machine-to-Machine type that refers to more sophisticated devices capable of capturing events related to any measurable industrial parameter and communicating the useful information to other devices or applications supported either in the same local network or in a remote cloud [2].

Figure 1 shows a prediction from the Forecasts and Trends report from Cisco Systems about the worldwide number of internet-connected devices that will incorporate into the IoT ecosystem in the next years [4]. To support radio-access for those new M2M devices, IoT-service providers are

deploying networks based on LPWAN (Low-Power Wide-Area Networks) technologies like NB-IoT, LTE-M, Sigfox, or LoRaWAN, all over the world. However, in remote rural zones of many countries, GPRS from 2G networks is still the only available radio-access technology for data services and probably will continue in that way for some years until the arrival of any LPWAN technology. Therefore, IoT application developers must consider GPRS access when designing end-devices for these zones. Fortunately, manufacturers of radio-communication modules provide options that support multi-radio-access technology (multi-RAT) connectivity to contemplate a future radio-access change in these areas.

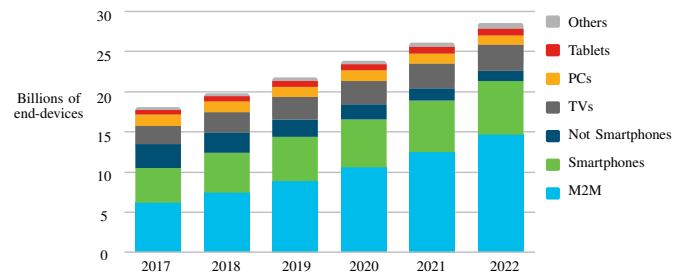


Fig. 1. Cisco Systems forecast on the number of connected M2M/IoT devices per year from 2017 to 2022 [4].

Table I shows the technical requirements for some of the M2M/IoT applications from Figure 1. The majority of them require low-data rates (at most, few hundreds of kbps), and very tolerant latency times. Typical GPRS deployments are capable to provide macrocell coverage in the range of few kms, with throughput rates up to 114 kbps and typical packet-delay in less than 1s [5].

The intended contribution of this paper is to share with the academic and developer community the implementation of a network prototype that recreates the GPRS air interface using a Software-Defined Radio (SDR) transceiver [9] controlled by computational processes implemented by the OpenBTS communication software [10]. The usage of Software-Defined Radio technology for prototyping has spread in recent years among the academic community due to its programmability and flexibility [11]. The vast majority of SDR transceivers can operate with frequencies up to 6 GHz and with bandwidths in the order of tens of MHz which enables the baseband processing of signal carriers corresponding to communication

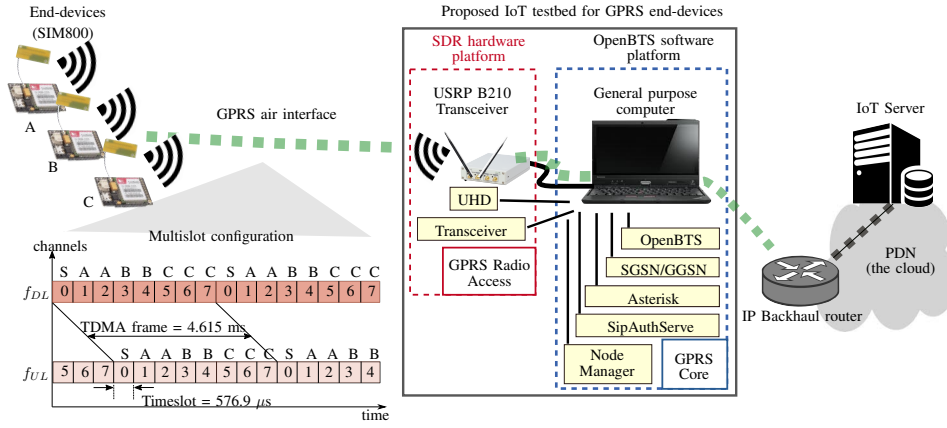


Fig. 2. Proposed GPRS-based IoT network testbed made of a combination of an SDR hardware and open-source software on a Linux Host.

TABLE I
M2M/IoT APPLICATIONS AND THEIR TECHNICAL REQUIREMENTS [2], [6], [7], [8].

Application	Throughput	Latency tolerance	Message Size	Mobility
Smart City	10s kbps - 10s Mbps	ms to seconds	bytes to kBs	Low
Smart Home	10s kbps - 10s Mbps	ms to seconds	bytes to kBs	Low
eHealth	10s - 100s kbps	ms	bytes to kBs	Low
Smart Metering	1 - 100s kbps	seconds	tens of bytes	Low
Smart Agriculture	10s - 100s kbps	seconds	tens of bytes	Low
Pollution Monitoring	10s bps - 10s kbps	seconds	bytes	Low
Asset Tracking	10s bps - 10s kbps	ms	bytes	High

systems such as FM and TV broadcasting, mobile networks, satellite communications, wireless-LAN networks, etc.

In the following sections, we'll describe the components of the prototype as well as its implementation, configuration and operation startup. At last, we'll perform two tests over the operation of the GPRS prototype network. The first one will analyse the captured signaling messages that an end-device exchanges with the GPRS prototype network when the end-device establishes a connection and sends data to an IoT server in the cloud. The second one will be about testing the GPRS network accessibility rate when various end-devices try to access the shared communication channel.

II. SYSTEM OVERVIEW.

The prototype shown in Figure 2 consists of a generic computer (Host) connected to an SDR peripheral. The host runs a Linux operating system than contains an implementation of the OpenBTS software platform. OpenBTS implements computational processes that emulate the tasks of a traditional GPRS network. OpenBTS uses the SDR-transceiver capabilities to create the GPRS radio interface, and uses the host TCP/IP protocol stack to create the backhaul network to allow the IP-packet routing towards to, or inwards from, external PDNs.

On the radio-access, OpenBTS uses the radio resource scheme found in [12], therefore, the communication is full-duplex with downlink (DL) and uplink (UL) channels on different frequencies. The end-devices share these channels using Time Division Multiple Access (TDMA) scheme. Each TDMA frame contains eight timeslots, one for signaling and

the others seven for user data. GPRS network allocates one or more timeslots to the end-devices based on both, the end-device traffic requirements and the multislot configuration enabled on the system. For example, in Figure 2, the network assigns two timeslots per TDMA frame to each user A and B, and three timeslots for user C.

At the back-end, OpenBTS implements several computational processes, called modules. OpenBTS module is the manager of the other modules and implements the GSM lower-layers processing. Transceiver module along the UHD driver supports on the air interface creation. SGSN/GGSN module is in charge of creating the GPRS tunnels that transports the end-device traffic with external PDNs. SipAuthServe module is in charge of the end-device registration in the network. The Node Manager module allows third party software to connect to OpenBTS and make use of certain functionalities. In this way, in the uplink path for instance, OpenBTS receives the radio-packets at the SDR device (which is controlled by the UHD driver and the Transceiver module), removes the radio-protocol headers, and encapsulates the payload into IP packets to deliver them to the correspondent PDN. The inverse process applies for the IP packets that arrives from external PDNs to the GPRS core network and target a certain end-device.

A. OpenBTS emulation of a GPRS Network Architecture.

OpenBTS is an open-source software platform that emulates the main capabilities of a GSM/GPRS network in the form of computational processes running on a generic computer[13]. Figure 3 shows a GPRS data connection over both, traditional

GPRS network architecture and OpenBTS architecture. In both cases, the end-device first establishes the Random Access procedure with the network[14].

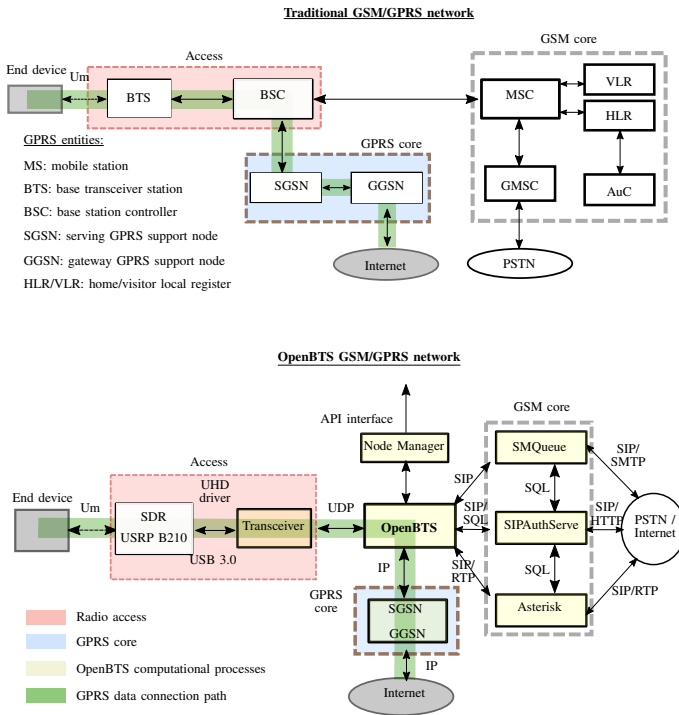


Fig. 3. Data connection path over traditional and OpenBTS GPRS networks.

On traditional architecture, BTS/BSC provides the radio-access while in the case of OpenBTS, it is the SDR peripheral controlled by the Transceiver module. Once the network allocates radio resources to the end-device, the latter requests the establishment of *Temporal Block Flows* (TBFs) to exchange radio-packets with network. With the DL and UL TBFs established, end-device triggers the *GPRS Attach* and *PDP Context Activation* procedures against the GPRS Core composed by SGSN and GGSN entities [15]. These procedures are mandatory in order to establish the data connection from a end-device with an external PDNs. SIPAuthServe module emulates the HLR functions (from traditional GPRS network) providing end-device registration and authentication functionalities. Also, the Node Manager process is a JSON-based API (application programming interface) to provide external management capabilities to third party software.

B. Software-Defined Radio and the USRP B210.

Software-Defined Radio (SDR) is a key transceiver technology for the emulation of mobile communication networks over general-purpose computers. The concept of SDR was conceived in the early 1990s [16] and it changed the radio communications design paradigms by leveraging the versatility provided by programmable software-reconfigurable hardware [17]. Figure 4 shows a generic SDR architecture replicated by the majority of commercially SDR devices. It consists of an analog front-end interfaced with ADC/DAC

converters, a Field Programmable Gate Array (FPGA), and a General-Purpose Processor (GPP).

The analog front-end is composed by bandpass and low-pass filters, and RF amplifiers to provide signal conditioning. Analog passband signals arrive at the receiver antenna(s) and are amplified via an LNA, then down-converted directly to baseband (or optionally to an Intermediate frequency (IF)), lowpass filtered, and finally, amplitude normalized by an automatic gain controller (AGC) before being sampled at the Nyquist rate by the ADC. The reverse process is followed on the front-end transmit chain, where incoming complex baseband signals from the DAC are filtered, up-converted, and amplified for passband transmission. Baseband and signal processing on the ADC output and DAC input digital samples is handled by the SDR back-end, which is usually composed of flexible FPGAs and GPPs processors[17].

USRP B210 is the SDR peripheral chosen for this implementation. The RF front-end, and ADC/DAC functionalities, are implemented on the radio frequency integrated chip (RFIC) AD9361 while the FPGA is a Xilinx Spartan 6. This RF board provides a 10mW max Tx-power that limits the coverage area. Therefore, the tests presented in section IV mainly focus on evaluating the radio-access behavior rather than coverage.

III. OPENBTS IMPLEMENTATION AND CONFIGURATION.

A. OpenBTS Implementation.

OpenBTS source code, hardware configuration, and installation procedures are provided by Range Networks [10],[18]. For the current implementation, it is used a generic computer with an 4th generation i7 AMD64 processor with 4GB of RAM, and 20GB of disk space, with Ubuntu 16.04, and with granted Internet connectivity.

B. OpenBTS Initialization and Configuration.

1) *Packet-routing setup*: OpenBTS uses Iptables module from the Linux kernel for creating routing rules to manage IP packets originated at the end-device towards external PDNs or vice-versa. Note that for the first command, `<if>` refers to the Host Ethernet or Wireless network interface.

```
$ sudo iptables -t nat -A POSTROUTING -o <if> -j MASQUERADE
$ sudo echo 1 > /proc/sys/net/ipv4/ip_forward
```

2) *OpenBTS Initialization*: Once networking is set up, the following commands are used to stop and start the OpenBTS services, and to access the command-line interface (CLI). These steps are useful when modifying a parameter that requires OpenBTS restarting.

```
openbts_systemd_scripts$ sudo ./openbts-stop
openbts_systemd_scripts$ sudo ./openbts-start
openbts_systemd_scripts$ ./openbts-cli
OpenBTS> _
```

3) *Network registration enabling*: To enable the network registration of any GPRS end-device:

```
OpenBTS> devconfig Control.LUR.OpenRegistration .*
```

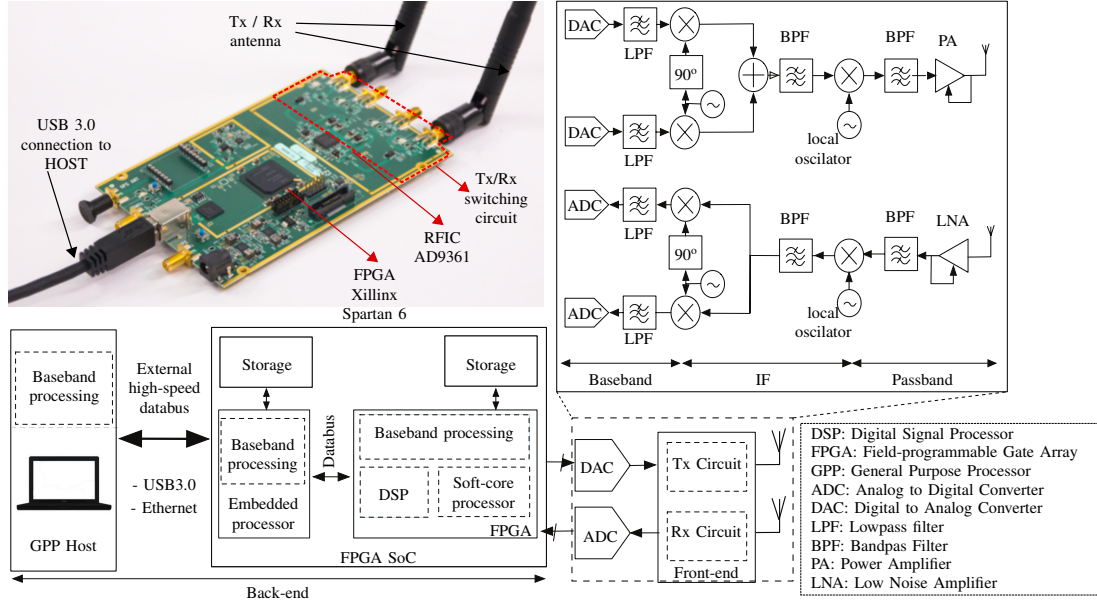


Fig. 4. SDR block diagram and the USRP B210 from Ettus Research [17].

4) *ARFCN channel configuration*: ARFCN stands for Absolute Radio-Frequency Channel Number which is a code that identifies certain uplink and downlink carrier. Current prototype network runs over ARFCN 512 in the DCS1800 band, which corresponds to carriers 1805.2MHz and 1710.2MHz for DL and UL channels, respectively.

```
OpenBTS> config GSM.Radio.Band 1800
OpenBTS> config GSM.Radio.CO 512
```

5) *Network ID parameters*: To specify the Mobile Country Code (MCC) and the Mobile Network Code (MNC) codes. For this paper, the end-devices use SIMcards with IMSI format 90170XXXXXXXXXX. Therefore, MCC and MNC are 901 and 70, respectively.

```
OpenBTS> config GSM.Identity.MCC 901
OpenBTS> config GSM.Identity.MNC 70
```

6) *GPRS service enabling*: By default, GPRS service is disabled on OpenBTS. To enable it on OpenBTS CLI:

```
OpenBTS> devconfig GPRS.Enable 1
```

7) *Resource allocation configuration*: By default, OpenBTS assigns all the 07 timeslots to SMS-and-voice services. Also, by default, it sets up the maximum number of timeslots per user as 03 for downlink and 02 for uplink. User must allow all user-data timeslots to be available for GPRS:

```
OpenBTS> devconfig GPRS.Channels.Min.CO 7
OpenBTS> devconfig GPRS.Multislot.Max.Downlink 2
OpenBTS> devconfig GPRS.Multislot.Max.Uplink 2
```

However, in section IV, these parameters are altered to find the best setup for multiple device connections.

8) *GGSN parameters configuration*: Specify a DNS server and disable the firewall filter:

```
OpenBTS> devconfig GGSN.DNS 8.8.8.8
OpenBTS> devconfig GGSN.Firewall.Enable 0
```

9) *Traffic capture*: To capture the signaling exchange between the end-device and OpenBTS with Wireshark [19]:

```
OpenBTS> devconfig Control.GSMTAP.GPRS 1
```

Additional configurations related to other OpenBTS functionalities can be found at the OpenBTS Manual [13].

C. Basic GPRS End-Device Implementation for tests.

Figure 5 shows the implementation of a GPRS end-device. It consists of a micro-controller Arduino Mega 2560 connected with a GSM/GPRS module SIM800L [20]. The user compiles the serial-connection code from [21], and use AT commands to perform step-by-step data connection against the OpenBTS GPRS network. [21] provides end-device source code to perform tests A and B from section IV.

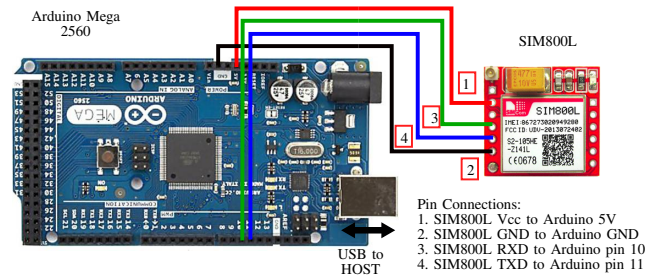


Fig. 5. Arduino Mega 2560 + SIM800L setup to implement a GPRS end-device controlled by a serial connection.

IV. TESTS PERFORMED

Figure 6 provides the schematics for testings. On the radio-access side, the network provides local IP addresses from the 192.168.99.0/24 network to the end-devices. On the backhaul side, the Host obtains an IP address from 10.0.113.1/24

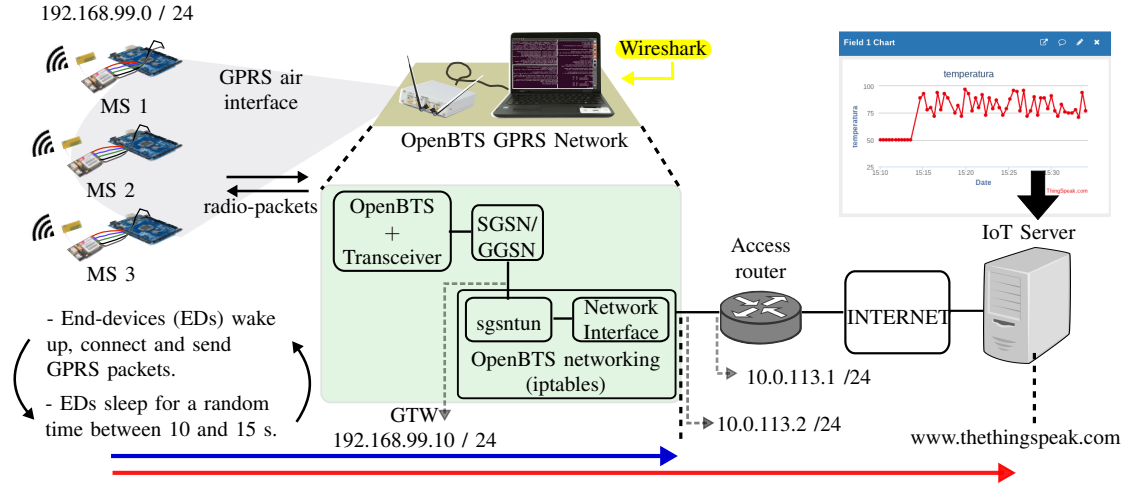


Fig. 6. Access availability and capacity test setup.

network provided by the Internet access router. To connect both networks, OpenBTS implements the virtual network interface `sgsntun` that logically interfaces with the Host physical network interface. Then, OpenBTS uses *Iptables* rules (section III-B1) to properly route the IP packets generated from the end-devices towards external PDNs, or vice versa.

A. IP data connection with IoT server

This test creates a GPRS data connection from an end-device against an IoT server in the cloud. Reference [22] provides the steps to implement a data collection channel on *TheThingSpeak* server. The end-device source code [21] uses the channel associated unique API key to establish a TCP data connection to send GET-type HTTP data packets to that channel. Wireshark application [19] is used to capture the signaling message exchange between the end-device and the IoT server, which Table II shows. It is identified the *Random Access* process, *Location Update* process, *TBF Establishment* process, and finally, the *Packet Transfer* process.

TABLE II
SIGNALING EXCHANGE BETWEEN END-DEVICE AND GPRS NETWORK.

Time	Protocol	Information	DL/UL
14.166	GSMTAP	(CCCH) (RR) Immediate Assignment	Downlink
14.335	LAPDm	U P, func=SABM(DTAP) (MM) Location Updating Request	Uplink
14.336	LAPDm	I, N(R)=0, N(S)=0(DTAP) (MM) Identity Request	Downlink
15.271	LAPDm	I, N(R)=1, N(S)=0(DTAP) (MM) Identity Response	Uplink
15.272	SIP	Request: REGISTER sip:127.0.0.1 (1 binding)	-
15.273	SIP	Status: 404 IMSI Not Found	-
16.353	LAPDm	I, N(R)=1, N(S)=2(DTAP) (MM) Location Updating Accept	Downlink
19.389	LAPDm	I, N(R)=1, N(S)=3(DTAP) (RR) Channel Release	Downlink
19.837	GSMTAP	(CCCH) (RR) Immediate Assignment	Downlink
23.342	GSMTAP	(CCCH) (RR) Immediate Assignment	Downlink
26.646	GSMTAP	(CCCH) (RR) Immediate Assignment	Downlink
29.938	GSMTAP	(CCCH) (RR) Immediate Assignment	Downlink
30.063	GSM RLC/MAC	GPRS UL:PACKET_RESOURCE_REQUEST	Uplink
30.065	GSM RLC/MAC	GPRS DL:GPRS_DL:PACKET_ULINK_ASSIGNMENT	Downlink
30.184	GSM RLC/MAC	GPRS UL:PACKET_CONTROL_ACKNOWLEDGEMENT	Uplink
30.189	GSM RLC/MAC	GPRS DL:GPRS_DL:PACKET_DOWNLINK_DUMMY_CONTROL_BLOCK	Downlink
30.189	GSM RLC/MAC	GPRS DL:GPRS_DL:PACKET_DOWNLINK_DUMMY_CONTROL_BLOCK	Downlink
30.203	GSM RLC/MAC	GPRS DL:GPRS_DL:PACKET_DOWNLINK_DUMMY_CONTROL_BLOCK	Downlink
30.267	GSM RLC/MAC	GPRS UL	Uplink
30.267	GSM RLC/MAC	GPRS UL	Uplink
30.282	GSM RLC/MAC	GPRS UL	Uplink
30.290	GSM RLC/MAC	GPRS DL:GPRS_DL:PACKET_DOWNLINK_DUMMY_CONTROL_BLOCK	Downlink
30.290	GSM RLC/MAC	GPRS DL:GPRS_DL:PACKET_DOWNLINK_DUMMY_CONTROL_BLOCK	Downlink
30.305	GSM RLC/MAC	GPRS DL:GPRS_DL:PACKET_DOWNLINK_ASSIGNMENT	Downlink
30.305	GSM RLC/MAC	GPRS DL:GPRS_DL:PACKET_DOWNLINK_DUMMY_CONTROL_BLOCK	Downlink
30.305	GSM RLC/MAC	GPRS DL	Downlink

The first message is a *Random Access Request* that the end-device sends to the network. Table II does not display this message because it is not pre-scheduled by the network. In response, network sends an *Immediate Assignment* message to specify the channel resources that the end-device can use for further signaling. Next, the end-device sends a *Location Update Request* message for network registry and authentication. The end-device sends an *Identity Response* message with its International Mobile Subscriber Identity (IMSI). In this case, the network does not recognize the end-device; however, the network will not restrict access given the parameters set up in section III-B3. After registering, the end-device releases the channel and enters into idle state.

Later on, the end-device triggers a new *Random Access* procedure, and at the fourth attempt, it gets the requested channel resources to perform DL and UL TBF Establishment for further GPRS data transfer. For that, the end-device sends a *Packet Resource Request* message that also contains a *GPRS Attach* and a *PDP Context Activation* request. The network issues a *Packet Uplink Assignment* message, and later on, it sends a *Packet Downlink Assignment* message. With that, the DL and UL TBFs are on. The following is the exchange of GPRS UL/DL messages, which contain upper protocol layers data as payload (such as HTTP, TCP, IP, etc.). This test successfully probes the concept of sending IP data packets towards an external IoT server on the Internet, through the implemented GPRS network.

B. Radio Access Availability Test.

Figure 6 also shows the setup for the radio-access availability test. Three end-devices attempt to establish GPRS connections with the implemented network. Once the connection is set up, the end-devices send 1024 Byte-payload ICMP packets towards a local IP address. This experiment tests the GPRS air interface segment and the internal-Host-networking segment, as depicted by the blue arrow in Figure 6. Once the end-device ends its transmission, it releases the allocated radio channels (timeslots) and gets into sleep mode. The end-device restarts

the data connection after a random interval of time no less than 10 seconds and no more than 15 seconds, and will repeat this process for 12 hours. This test considers scenarios in which 01, 02, and 03 end-devices use the radio-access simultaneously.

TABLE III
MULTISLOT CONFIGURATION PARAMETERS.

Config.	GPRS.Channels. Min.C0	GPRS.Multislot. Max.Uplink	GPRS.Multislot. Max.Downlink
1	7	1	1
2	7	2	2
3	6	1	1
4	6	2	2

As described in Figure 2, the multislot configuration allows the network to allocate more than 01 timeslot per TDMA frame to an arbitrary end-device. This test is about measuring and comparing the network radio-access availability for each case of Table III. Once the system is set up with a multislot configuration (section III-B7) from Table III, the end-devices will perform random data sending for 12 hours. The Arduino program [9] on each end-device computes successful and lost ICMP packets allowing to compute the radio-access availability rate for each end-device. This test is repeated 03 times for each multislot configuration. Figure 7 shows the success rate for each test scenario (01, 02, 03 connected devices). The graph shows that success rate for multi-slot configuration 2 (GPRS.Channels.Min.C0 7, GPRS.Multislot.Max.Downlink 2, GPRS.Multislot.Max.Uplink 2) is the best configuration for radio-access availability with multi-device connections.

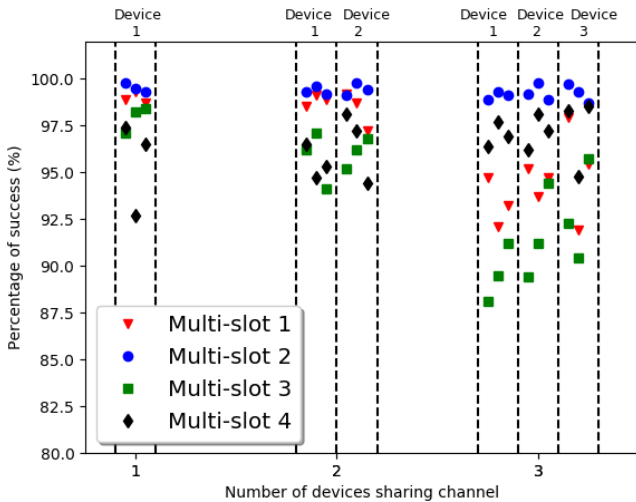


Fig. 7. Comparison of success rates for each multislot configuration on 03 scenarios.

V. CONCLUSIONS

A prototype GPRS network is implemented for academic purposes to test GPRS-based IoT applications with the use of open-source software (OpenBTS) and SDR hardware (USRP

B210). It provides GPRS network access to IoT end-devices and allows them to implement end-to-end data transfer to an IoT server in the cloud. The signaling message exchanging between end-devices and prototype network corresponds to a traditional GPRS network signaling and also it was found a multislot configuration to allow a suitable accessibility for multiple-device connections.

REFERENCES

- [1] P. Wang, R. Valerdi, S. Zhou, and L. Li, "Introduction: Advances in IoT research and applications," *Information Systems Frontiers*, vol. 17, no. 2, pp. 239–241, Apr 2015.
- [2] C. Anton-Haro and M. Dohler, *Machine-to-machine (M2M) communications: architecture, performance and applications*. Elsevier, 2014.
- [3] R. Y. Rodriguez, M. R. Julcapoma, and R. A. Jacinto, "Network monitoring environmental quality in agriculture and pisciculture with low power sensor nodes based on zigbee and gprs technology," in *2016 IEEE XXIII International Congress on Electronics, Electrical Engineering and Computing (INTERCON)*. IEEE, 2016, pp. 1–6.
- [4] Cisco Systems, "Cisco Annual Internet Report, 2018–2023," 2018.
- [5] D. Robinson, "NB-IoT (LTE Cat-NB1/Narrow-band IoT) performance evaluation of variability in multiple LTE vendors, UE devices and MNOs," Ph.D. dissertation, Stellenbosch University, 2019.
- [6] P. K. Verma, R. Verma, A. Prakash, A. Agrawal, K. Naik, R. Tripathi, M. Alsabaan, T. Khalifa, T. Abdelkader, and A. Abogharaf, "Machine-to-Machine (M2M) communications: A survey," *Journal of Network and Computer Applications*, vol. 66, pp. 83–105, 2016.
- [7] S. Persia and L. Rea, "Next generation M2M cellular networks: LTE-MTC and NB-IoT capacity analysis for smart grids applications," in *2016 AEIT International Annual Conference (AEIT)*, Oct 2016, pp. 1–6.
- [8] J. Finnegan and S. Brown, "A comparative survey of LPWA networking," *arXiv:1802.04222*, 2018.
- [9] Ettus Research. (2019) USRP B210. [Online]. Available: <https://www.ettus.com/product/details/UB210-KIT>
- [10] Range Networks. (2019) OpenBTS. [Online]. Available: <https://github.com/RangeNetworks/dev>
- [11] R. Akeela and B. Dezfouli, "Software-defined radios: Architecture, state-of-the-art, and challenges," *Computer Communications*, vol. 128, pp. 106–125, 2018.
- [12] *Digital cellular telecommunications system (Phase 2+); Multiplexing and Multiple Access on the Radio Path*, 3GPP - ETSI, 2003, v8.11.0.
- [13] M. Iedema, *Getting Started with OpenBTS: Build Open Source Mobile Networks*. "O'Reilly Media, Inc.", 2014.
- [14] *Digital cellular telecommunications system (Phase 2+); General Packet Radio Service (GPRS); Overall description of the GPRS radio interface; Stage 2*, 3GPP - ETSI, 2004, v8.12.0.
- [15] *Digital cellular telecommunications system (Phase 2+); General Packet Radio Service (GPRS) Service description; Stage 2*, 3GPP - ETSI, 2002, v7.9.0.
- [16] J. Mitola, "Software radio architecture: a mathematical perspective," *IEEE Journal on selected areas in communications*, vol. 17, no. 4, pp. 514–538, 1999.
- [17] G. Sklivanitis, A. Gannon, S. Batalama, and D. Pados, "Addressing next-generation wireless challenges with commercial software-defined radio platforms," *IEEE Communications Magazine*, vol. 54, 01 2016.
- [18] Luis Gabriel Yaya Garcia. (2019) Instalar OpenBTS en Ubuntu 16.04 GNURadio. [Online]. Available: <https://github.com/luizener/Install-openbts/wiki/Instalar-OpenBTS-en-Ubuntu-16.04-GNURadio>
- [19] Wireshark. (2019) Wireshark home page. [Online]. Available: <https://www.wireshark.org/>
- [20] SIMCom. (2016) SIM800 Hardware Design, v1.09.
- [21] Oscar Llerena. (2019) OpenBTS GPRS tests. [Online]. Available: https://github.com/blinkomaniak/openbts_GPRS_tests
- [22] MathWorks. (2019) Collect data in a new channel. [Online]. Available: <https://www.mathworks.com/help/thingspeak/collect-data-in-a-new-channel.html>